

Case Study: Netflix Schema Design

Problem Statement

Design Database Schema for a system like Netflix with following Use Cases.

Use Cases

1. Netflix has **users**.
2. Every **user** has an email and a password.
3. Users can create **profiles** to have separate independent environments.
4. Each **profile** has a name and a type. **Type** can be KID or ADULT.
5. There are multiple **videos** on netflix.
6. For each **video**, there will be a title, description and a cast.
7. A cast is a list of actors who were a part of the video. For each **actor** we need to know their name and list of videos they were a part of.
8. For every video, for any profile who watched that video, we need to know the status (COMPLETED/ IN PROGRESS).
9. For every profile for whom a video is in progress, we want to know their last watch timestamp.

Tables.

Users

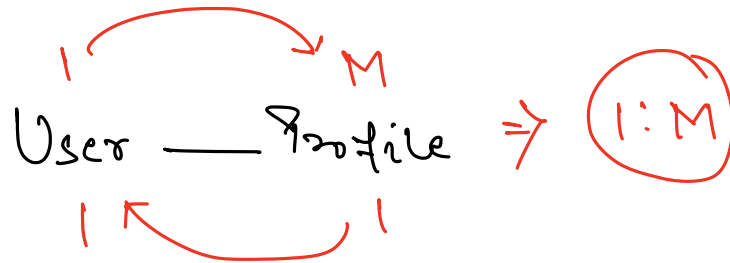
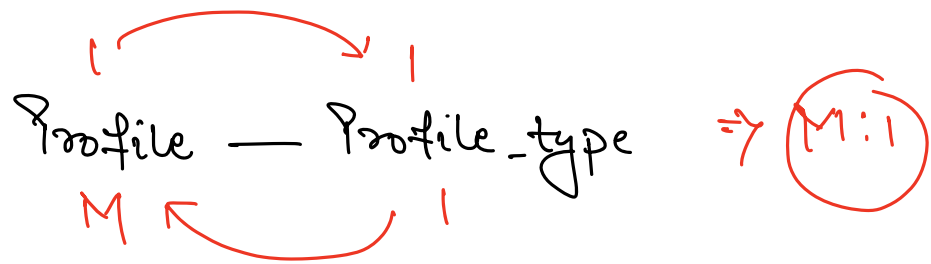
- id (PK)
- name
- email
- password

profiles

- id (PK)
- name
- profile-type-id
- user-id.

profile_type

- id
- Value

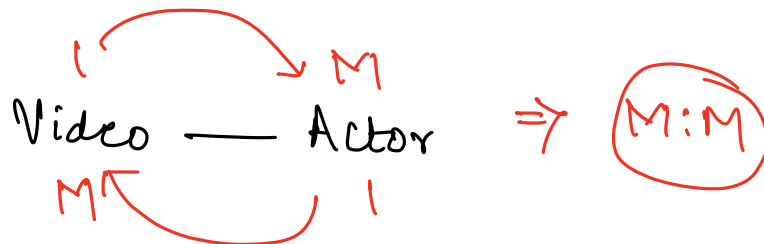


Videos

- id (PK)
- title
- description

actors

- id (PK)
- name
- age



Videos_actors

Video.id	actor.id



Videos_profiles

Video-id	profile-id	status	timestamp.

Case Study: Employee Management System

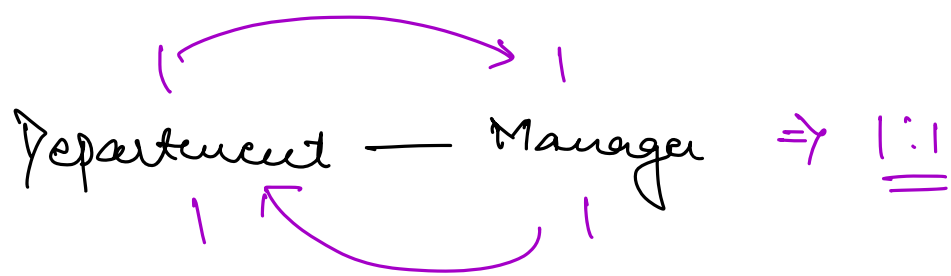
1. The company has **employees** with employee code, name, email, phone, date of joining, and **status** - ACTIVE or INACTIVE.
2. **Departments** exist with name, cost center, and a current manager who is also an employee.
3. Each employee belongs to exactly one department at a time. Department membership must retain history with from and to dates.
4. **Roles** define job functions. Employees can hold multiple roles over time with from and to dates and a primary-role flag during an interval.
5. **Compensation** must be versioned - basic, HRA, allowances, bonus - with effective_from dates. **Never overwrite history.**
6. **Attendance** is stored per employee per date with **work type** - REGULAR, OVERTIME, WFH, LEAVE - and hours.
7. **Leave requests** store type, start and end dates, approval status, and approver - an employee.
8. **Assets** - laptops or phones - have type and serial number. Assets are issued to employees with issued_on and returned_on dates. Only issued assets appear in the assignment table.
9. **Performance reviews** occur per cycle with reviewer - an employee - rating, comments, and submission time.
10. **Access groups** model application access. Employees can be added or removed from groups with start and end dates.

→ employees.

- id (PK)
- emp-code
- name
- email
- phone --
- **current-dept-id**

→ departments

- id
- name
- Cost-center
- manager-id

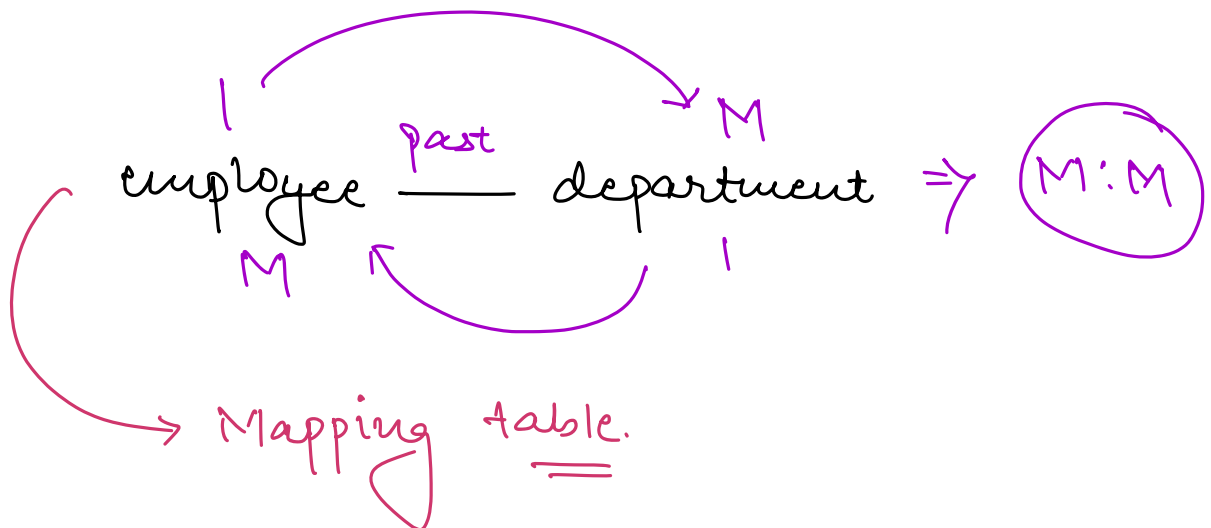
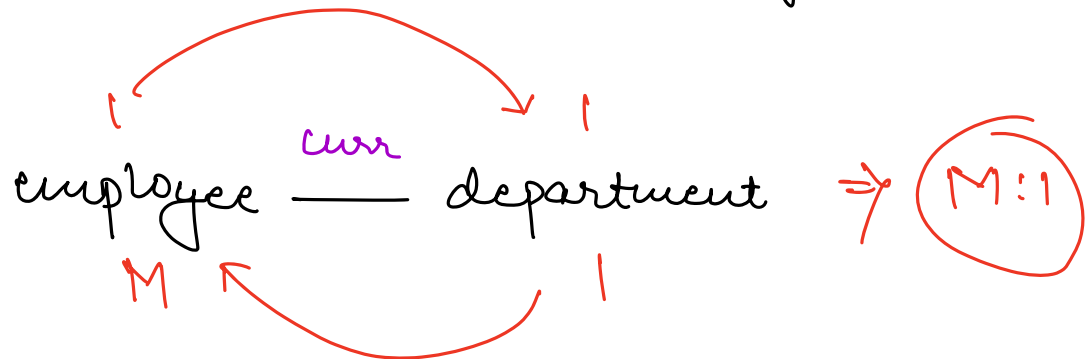


roles

→ id
→ name

compensation

→ Comp-id
→ basic
→ hra
→ allow -
→ bonus
→ effective-from



emp_dept_history

emp-id	dept-id	Start	end
104	10		

Banks DB.

accounts

id	number	name	balance
—	—	A	10,000
—	—	B	5,000
—	—	C	7000

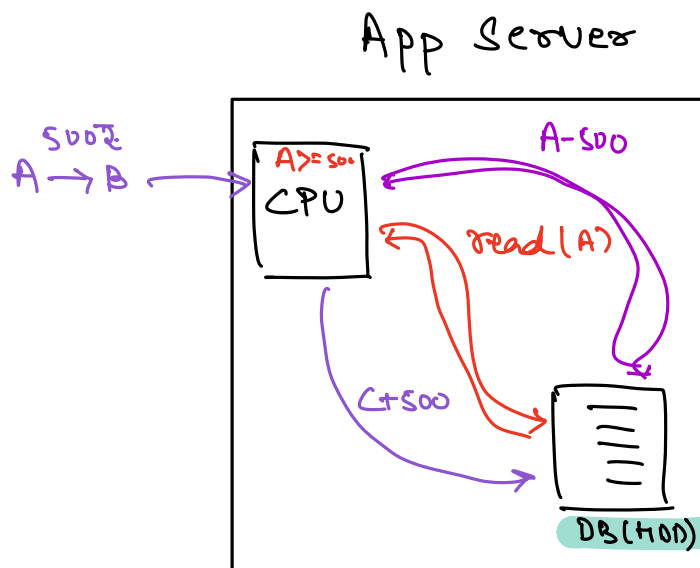
Transfer: $A \xrightarrow{500\text{₹}} B$

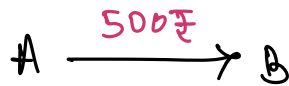
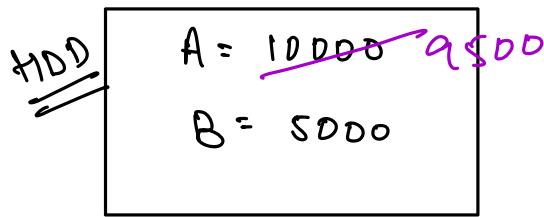
I) Read balance of A $\Rightarrow X \Rightarrow$ DB Operation

II) $X \geq 500$

III) Debit balance from A $\Rightarrow$ DB Operation

IV) Credit balance to B $\Rightarrow$ DB Operation





bank_transfer($\textcircled{A}$, $\textcircled{B}$, 500) {

1) $x \leftarrow \text{Read}(A)$

2) if ($x \geq \text{amount}$) {

3) $x = x - 500$

4) $A \leftarrow \text{write}(x)$

}

5) $x \leftarrow \text{Read}(B)$

XXX $\xrightarrow{\hspace{10em}}$

6) $x = x + 500$

7) $B \leftarrow \text{write}(x)$

}

Transaction : A set of logically grouped statements to perform a particular task.

$\downarrow$

Either a transaction should execute completely $\textcircled{\text{or}}$ it shouldn't execute at all.

$$\begin{array}{l}
 A = \cancel{12000} \quad \underline{\underline{9500}} \\
 B = \cancel{5000} \quad \underline{\underline{5500}} \\
 C = \cancel{4000} \quad \underline{\underline{6600}}
 \end{array}$$

$$\underline{\underline{5400}}$$

T1

$$A \xrightarrow{500\text{€}} B$$

T2

$$C \xrightarrow{400\text{€}} B$$

- 1) ¹⁰⁰⁰⁰ $X \leftarrow \text{Read}(A)$
- 2) $X \geq 500$ ✓
- 3) ⁹⁵⁰⁰ $X = X - 500$
- 7) $A \leftarrow \underline{\underline{\text{Write}(X)}}$
- 9) ⁵⁰⁰⁰ $X \leftarrow \text{Read}(B)$
- 10) ⁵⁵⁰⁰ $X \leftarrow X + 500$
- 12) $B \leftarrow \underline{\underline{\text{Write}(X)}}$

- 4) ⁹⁰⁰⁰ $X \leftarrow \text{Read}(C)$
- 5) $X \geq 400$ ✓
- 6) ⁶⁶⁰⁰ $X = X - 400$
- 8) $C \leftarrow \underline{\underline{\text{Write}(X)}}$
- 11) ⁵⁰⁰⁰ $X \leftarrow \text{Read}(B)$
- 13) ⁵⁴⁰⁰ $X \leftarrow X + 400$
- 14) $\underline{\underline{B \leftarrow \underline{\underline{\text{Write}(X)}}}}$

Atomicity
Consistency → Correctness.
=

Isolation

Durability.